



AUREON RECONCILIATION ENGINE — FOCUSED DEEP DIVE

Selected Priority Sections:

1. **SECTION 7:** 12-Week Phased Roadmap (with concrete deliverables & success metrics)
2. **SECTION 8:** Data Model & CTM Refinements (for your existing PostgreSQL schema)
3. **SECTION 9:** Integration Architecture (rule engine ↔ ingestion layer ↔ LLM gateway)

SECTION 7 — PRIORITIZED 12-WEEK IMPLEMENTATION ROADMAP

7.0 Pre-Roadmap: Existing State Assessment

Current Aureon Stack (as of today):

- Backend: FastAPI + PostgreSQL (multi-tenant, tenant_id on all core tables)
- Ingestion: LLM-based schema mapping (Gemini 2.5 Flash) → normalized tables (broker_trades, bank_txns, holdings, nav_logs, corp_actions, etc.)
- AI Layer: OpenAI + Gemini abstracted via llm_gateway.py
- Frontend: React/Vite on Vercel
- Deployment: Cloud-hosted (Render), multi-tenant SaaS model

What already exists:

- Table structure for trades, cash, positions, NAV, corporate actions
- File ingestion pipeline (PDF, XLSX, CSV parsing)
- Basic LLM integration for schema mapping
- No reconciliation rule engine yet

What will be built:

- Deterministic rule engine (Tier-1 & Tier-2 rules in SQL/Python)
- Exception management workflows (detection, escalation, manual override, learning)
- Self-healing logic (pattern detection, rule tuning)
- Reporting dashboards (breaks by domain, age, severity)

7.1 PHASE 1: MVP (WEEKS 1-4) — Core Trade ↔ Cash + Basic NAV

Objective: Deploy working reconciliation for **India PMS, equity-only, T+1 settlement**, covering 70% of typical breaks with minimal ops burden.

Scope:

- Trade ↔ Cash reconciliation (exact match + key tolerances)
- Basic NAV sanity checks (AUM sum, NAV per unit calc)
- Exception detection & escalation workflow
- Manual override logging
- No AI/fuzzy logic yet (Tier-1 & 2 only)

7.1.1 Rules to Implement (Priority Order)

Rule ID	Domain	Name	Complexity	Estimated Impact
TRADE_CASH_001	Trade ↔ Cash	Exact amount match	LOW	80% of breaks
TRADE_CASH_005	Trade ↔ Cash	Trade status ↔ cash status	LOW	5% of breaks
TRADE_CASH_006	Trade ↔ Cash	Settlement date variance	LOW	10% of breaks
TRADE_CASH_008	Trade ↔ Cash	Buy/sell sign validation	LOW	2% of breaks
TRADE_CASH_003	Trade ↔ Cash	Fee decomposition (STT/GST)	MEDIUM	3% of breaks
NAV_VAL_001	NAV	Holdings sum = AUM	LOW	90% of NAV breaks
NAV_VAL_002	NAV	NAV per unit calc	LOW	8% of NAV breaks
NAV_VAL_003	NAV	Internal vs custodian NAV	MEDIUM	2% of NAV breaks
SUS_001	Suspense	Unmatched cash detection	LOW	Foundation for exception queue
SUS_007	Suspense	Break aging report	LOW	Ops reporting

Total: 10 rules covering ~90% of breaks in Phase 1 scope.

7.1.2 Deliverables

Week 1:

- └ Design: Finalize data model (Section 8 below)
- └ Design: Exception schema (exceptions table, status enum, severity levels)
- └ Setup: Rule definitions table (rule_id, logic, tolerances, versioning)
- └ Setup: Learning events table (for future self-healing)
- └ Code: Skeleton of ReconciliationRuleEngine class

Week 2:

- └─ Implement: TRADE_CASH_001 (exact match) + unit tests
- └─ Implement: TRADE_CASH_005 (status alignment) + unit tests
- └─ Implement: TRADE_CASH_006 (timing variance) + unit tests
- └─ Implement: NAV_VAL_001 (AUM sum) + unit tests
- └─ Integration: Wire rule engine to daily batch job

Week 3:

- └─ Implement: TRADE_CASH_003 (fee decomposition) + India-specific STT/GST logic
- └─ Implement: NAV_VAL_002 (NAV per unit) + TRADE_CASH_008 (buy/sell sign)
- └─ Implement: Exception detection + escalation queue
- └─ Implement: Manual override logging (learning_events insert)
- └─ Test: End-to-end flow with test data (10-20 PMS trades)

Week 4:

- └─ Implement: Reporting (exception dashboard, break aging)
- └─ UI: Add "Reconciliation" tab to frontend (exception queue, manual fix form)
- └─ Ops workflow: Exception notification (Slack integration)
- └─ QA: Full integration testing (with real custodian/broker files)
- └─ Documentation: Ops runbook (how to investigate breaks, manual fixes)
- └─ Soft launch: Deploy to staging; run for 1 week on historical data

7.1.3 Success Metrics

Metric	Target	How Measured
% of trades auto-reconciled	≥85%	(matched_trades / total_trades)
Average time-to-resolution (ops)	<4 hours for CRITICAL breaks	Timestamp from escalation to closure
False positive rate	<5%	(auto-resolved_but_later_found_wrong / auto_resolved)
Mean break aging	<1 day	(sum of break age / break count)
Ops efficiency gain	4-6 hours saved/day/person	Survey ops team
NAV variance detection	Catches 100% of >1% NAV variances	Test with known variance scenarios

7.1.4 Phase 1 Tech Stack

```
# backend/rule_engine/phase1_engine.py

from fastapi import APIRouter, HTTPException
from sqlalchemy import create_engine, text
import logging
from datetime import datetime, timedelta
from enum import Enum

class SeverityLevel(str, Enum):
    CRITICAL = "CRITICAL"
```

```
HIGH = "HIGH"
MEDIUM = "MEDIUM"
LOW = "LOW"
```

```
class BreakStatus(str, Enum):
    OPEN = "OPEN"
    ESCALATED = "ESCALATED"
    IN_INVESTIGATION = "IN_INVESTIGATION"
    RESOLVED = "RESOLVED"
    CLOSED = "CLOSED"
```

```
class Phase1ReconciliationEngine:
```

```
    def __init__(self, db_session, tenant_id: str, as_of_date: datetime.date):
        self.db = db_session
        self.tenant_id = tenant_id
        self.as_of_date = as_of_date
        self.logger = logging.getLogger(f"recon_engine_{tenant_id}")
        self.exceptions_created = []
```

```
    def run_phase1_daily_reconciliation(self) -> dict:
```

```
        """
        Main orchestrator for Phase 1 reconciliation.
        Returns: summary of matched/broken reconciliations.
        """
```

```
        summary = {
            "run_date": self.as_of_date,
            "tenant_id": self.tenant_id,
            "total_trades": 0,
            "matched_trades": 0,
            "breaks_detected": 0,
            "breaks_by_severity": {},
            "execution_time_seconds": 0
        }
```

```
        start_time = time.time()
```

```
        try:
```

```
            # Load data for reconciliation date
            trades = self._load_settled_trades(self.as_of_date)
            bank_txns = self._load_bank_txns(self.as_of_date)
            holdings = self._load_holdings(self.as_of_date)
```

```
            summary["total_trades"] = len(trades)
```

```
            # ===== RULE: TRADE_CASH_001 (Exact Match) =====
```

```
            self.logger.info(f"Running TRADE_CASH_001 for {len(trades)} trades...")
            matched, breaks = self._reconcile_trade_cash_exact(trades, bank_txns)
            summary["matched_trades"] += len(matched)
            summary["breaks_detected"] += len(breaks)
            self._log_exceptions(breaks, rule_id="TRADE_CASH_001", severity=SeverityLevel)
```

```
            # ===== RULE: TRADE_CASH_005 (Status Alignment) =====
```

```
            self.logger.info(f"Running TRADE_CASH_005...")
            breaks_status = self._validate_trade_cash_status_alignment(trades, bank_txns)
            summary["breaks_detected"] += len(breaks_status)
            self._log_exceptions(breaks_status, rule_id="TRADE_CASH_005", severity=SeverityLevel)
```

```

# ===== RULE: TRADE_CASH_006 (Timing) =====
self.logger.info(f"Running TRADE_CASH_006...")
breaks_timing = self._validate_settlement_timing(trades, bank_txns)
summary["breaks_detected"] += len(breaks_timing)
self._log_exceptions(breaks_timing, rule_id="TRADE_CASH_006", severity=Severity)

# ===== RULE: TRADE_CASH_008 (Sign) =====
self.logger.info(f"Running TRADE_CASH_008...")
breaks_sign = self._validate_buy_sell_direction(trades, bank_txns)
summary["breaks_detected"] += len(breaks_sign)
self._log_exceptions(breaks_sign, rule_id="TRADE_CASH_008", severity=Severity)

# ===== RULE: TRADE_CASH_003 (Fees) =====
self.logger.info(f"Running TRADE_CASH_003...")
breaks_fees = self._validate_fee_decomposition(trades)
summary["breaks_detected"] += len(breaks_fees)
self._log_exceptions(breaks_fees, rule_id="TRADE_CASH_003", severity=Severity)

# ===== RULE: NAV_VAL_001 (AUM Sum) =====
self.logger.info(f"Running NAV_VAL_001...")
nav_breaks = self._reconcile_aum_sum(holdings)
summary["breaks_detected"] += len(nav_breaks)
self._log_exceptions(nav_breaks, rule_id="NAV_VAL_001", severity=SeverityLevel)

# ===== RULE: NAV_VAL_002 (NAV Per Unit) =====
self.logger.info(f"Running NAV_VAL_002...")
nav_unit_breaks = self._validate_nav_per_unit()
summary["breaks_detected"] += len(nav_unit_breaks)
self._log_exceptions(nav_unit_breaks, rule_id="NAV_VAL_002", severity=Severity)

# Summarize by severity
summary["breaks_by_severity"] = self._count_exceptions_by_severity()

# Escalate to ops queue
self._escalate_exceptions_to_ops()

# Generate reports
self._generate_break_summary_report()

except Exception as e:
    self.logger.error(f"Reconciliation run failed: {e}", exc_info=True)
    summary["error"] = str(e)

finally:
    summary["execution_time_seconds"] = time.time() - start_time
    self.logger.info(f"Reconciliation complete: {summary}")

return summary

# ===== RULE IMPLEMENTATIONS =====

def _reconcile_trade_cash_exact(self, trades: list, bank_txns: list) -> tuple:
    """
    RULE TRADE_CASH_001: For each SETTLED trade, find matching cash entry.
    Returns: (matched_count, breaks_list)

```

```

"""
matched = []
breaks = []

for trade in trades:
    if trade.status != "SETTLED":
        continue

    # Find matching cash
    candidate_cash = [
        txn for txn in bank_txns
        if (
            abs(txn.amount - trade.net_amount) < 1.0 # Exact match (1 rupee tolerance)
            and txn.currency == trade.settlement_currency
            and trade.settlement_date <= txn.value_date <= trade.settlement_date
        )
    ]

    if len(candidate_cash) == 1:
        matched.append({
            "trade_id": trade.id,
            "cash_id": candidate_cash[0].id,
            "matched_amount": candidate_cash[0].amount,
            "matched_date": candidate_cash[0].value_date
        })
        # Mark in DB: update matched_cash_id on trade record
        self.db.execute(
            text("""
                UPDATE broker_trades
                SET matched_cash_id = :cash_id, matched_timestamp = NOW()
                WHERE id = :trade_id
            """),
            {"cash_id": candidate_cash[0].id, "trade_id": trade.id}
        )
    elif len(candidate_cash) > 1:
        breaks.append({
            "trade_id": trade.id,
            "rule_id": "TRADE_CASH_001",
            "break_type": "MULTIPLE_CASH_CANDIDATES",
            "candidates": [c.id for c in candidate_cash],
            "severity": SeverityLevel.HIGH,
            "message": f"Trade {trade.id} has {len(candidate_cash)} matching cash"
        })
    else:
        breaks.append({
            "trade_id": trade.id,
            "rule_id": "TRADE_CASH_001",
            "break_type": "NO_MATCHING_CASH",
            "expected_amount": trade.net_amount,
            "expected_currency": trade.settlement_currency,
            "expected_date_range": (trade.settlement_date, trade.settlement_date),
            "severity": SeverityLevel.CRITICAL,
            "message": f"No matching cash for settled trade {trade.id}"
        })

return matched, breaks

```

```

def _validate_trade_cash_status_alignment(self, trades: list, bank_txns: list) -> list:
    """
    RULE TRADE_CASH_005: If trade.status == SETTLED, cash.status must also be SETTLED
    """
    breaks = []

    for trade in trades:
        if trade.status != "SETTLED":
            continue

        # Find matched cash
        matched_cash = [txn for txn in bank_txns if txn.id == trade.matched_cash_id]

        if matched_cash:
            cash = matched_cash[0]
            if cash.status not in ["POSTED", "SETTLED", "CONFIRMED"]:
                breaks.append({
                    "trade_id": trade.id,
                    "cash_id": cash.id,
                    "rule_id": "TRADE_CASH_005",
                    "break_type": "STATUS_MISMATCH",
                    "trade_status": trade.status,
                    "cash_status": cash.status,
                    "severity": SeverityLevel.CRITICAL,
                    "message": f"Trade {trade.id} is SETTLED but cash {cash.id} is {cash.status}"
                })

    return breaks

def _validate_settlement_timing(self, trades: list, bank_txns: list) -> list:
    """
    RULE TRADE_CASH_006: Settlement should be ±2 days of expected settlement_date.
    """
    breaks = []

    for trade in trades:
        if not trade.matched_cash_id:
            continue

        cash = next((txn for txn in bank_txns if txn.id == trade.matched_cash_id), None)
        if not cash:
            continue

        days_late = (cash.value_date - trade.settlement_date).days

        if days_late < 0:
            breaks.append({
                "trade_id": trade.id,
                "cash_id": cash.id,
                "rule_id": "TRADE_CASH_006",
                "break_type": "EARLY_SETTLEMENT",
                "days_early": abs(days_late),
                "severity": SeverityLevel.MEDIUM,
                "message": f"Cash posted {abs(days_late)} days before settlement_date"
            })

```

```

        elif days_late > 2:
            breaks.append({
                "trade_id": trade.id,
                "cash_id": cash.id,
                "rule_id": "TRADE_CASH_006",
                "break_type": "LATE_SETTLEMENT",
                "days_late": days_late,
                "severity": SeverityLevel.MEDIUM,
                "message": f"Cash posted {days_late} days after settlement_date"
            })

    return breaks

def _validate_buy_sell_direction(self, trades: list, bank_txns: list) -> list:
    """
    RULE TRADE_CASH_008: BUY trades should have negative amount (debit), SELL positive
    """
    breaks = []

    for trade in trades:
        if not trade.matched_cash_id:
            continue

        cash = next((txn for txn in bank_txns if txn.id == trade.matched_cash_id), None)
        if not cash:
            continue

        # Convention: BUY (debit) = negative cash, SELL (credit) = positive cash
        expected_sign = -1 if trade.side == "BUY" else 1
        actual_sign = 1 if cash.amount > 0 else -1

        if expected_sign != actual_sign:
            breaks.append({
                "trade_id": trade.id,
                "cash_id": cash.id,
                "rule_id": "TRADE_CASH_008",
                "break_type": "BUY_SELL_SIGN_MISMATCH",
                "trade_side": trade.side,
                "cash_amount": cash.amount,
                "severity": SeverityLevel.HIGH,
                "message": f"Trade is {trade.side} but cash amount is {cash.amount}"
            })

    return breaks

def _validate_fee_decomposition(self, trades: list) -> list:
    """
    RULE TRADE_CASH_003: Validate gross - fees = net. India-specific: STT, GST.
    """
    breaks = []

    for trade in trades:
        # For India: STT applies only on SELL
        expected_stt = 0
        if trade.side == "SELL":
            expected_stt = trade.gross_amount * 0.001 # 0.1% as of 2024

```

```

        if trade.stt and abs(trade.stt - expected_stt) > 0.5:
            breaks.append({
                "trade_id": trade.id,
                "rule_id": "TRADE_CASH_003",
                "break_type": "STT_MISMATCH",
                "expected_stt": expected_stt,
                "actual_stt": trade.stt,
                "severity": SeverityLevel.MEDIUM,
                "message": f"STT mismatch: expected {expected_stt}, got {trade.stt}"
            })

        # GST = 18% on (brokerage + exchange_fee), NOT on STT
        eligible_for_gst = trade.brokerage + trade.exchange_fee
        expected_gst = eligible_for_gst * 0.18

        if trade.gst and abs(trade.gst - expected_gst) > 0.5:
            breaks.append({
                "trade_id": trade.id,
                "rule_id": "TRADE_CASH_003",
                "break_type": "GST_MISMATCH",
                "expected_gst": expected_gst,
                "actual_gst": trade.gst,
                "severity": SeverityLevel.MEDIUM,
                "message": f"GST mismatch: expected {expected_gst}, got {trade.gst}"
            })

        # Verify: gross - brokerage - stt - exchange_fee - gst = net
        expected_net = trade.gross_amount - (trade.brokerage or 0) - (trade.stt or 0)

        if abs(trade.net_amount - expected_net) > 1.0:
            breaks.append({
                "trade_id": trade.id,
                "rule_id": "TRADE_CASH_003",
                "break_type": "NET_AMOUNT_MISMATCH",
                "expected_net": expected_net,
                "actual_net": trade.net_amount,
                "variance": abs(trade.net_amount - expected_net),
                "severity": SeverityLevel.MEDIUM,
                "message": f"Net amount mismatch: expected {expected_net}, got {trade.net_amount}"
            })

    return breaks

def _reconcile_aum_sum(self, holdings: list) -> list:
    """
    RULE NAV_VAL_001: AUM = sum(holdings.total_value) + cash_balance.
    """
    breaks = []

    # Calculate AUM
    holdings_value = sum(h.total_value for h in holdings)
    cash_balance = self._get_cash_balance(self.as_of_date)
    calculated_aum = holdings_value + cash_balance

    # Fetch reported AUM from nav_logs

```

```

nav_log = self.db.query(text("""
    SELECT aum FROM nav_logs
    WHERE tenant_id = :tenant_id AND nav_date = :nav_date AND source = 'INTERNAL'
    LIMIT 1
    """), {"tenant_id": self.tenant_id, "nav_date": self.as_of_date}).fetchone()

if nav_log:
    reported_aum = nav_log.aum
    variance_pct = abs(calculated_aum - reported_aum) / reported_aum if reported_aum

    if variance_pct > 0.005: # 0.5% tolerance
        breaks.append({
            "rule_id": "NAV_VAL_001",
            "break_type": "AUM_MISMATCH",
            "calculated_aum": calculated_aum,
            "reported_aum": reported_aum,
            "variance_abs": abs(calculated_aum - reported_aum),
            "variance_pct": variance_pct * 100,
            "holdings_value": holdings_value,
            "cash_balance": cash_balance,
            "severity": SeverityLevel.CRITICAL,
            "message": f"AUM mismatch: calculated {calculated_aum}, reported {reported_aum}"
        })

return breaks

def _validate_nav_per_unit(self) -> list:
    """
    RULE NAV_VAL_002: NAV per unit = AUM / total_units.
    """
    breaks = []

    nav_log = self.db.query(text("""
        SELECT nav_per_unit, aum FROM nav_logs
        WHERE tenant_id = :tenant_id AND nav_date = :nav_date AND source = 'INTERNAL'
        LIMIT 1
        """), {"tenant_id": self.tenant_id, "nav_date": self.as_of_date}).fetchone()

    if nav_log:
        total_units = self._get_total_units()
        expected_nav = nav_log.aum / total_units if total_units > 0 else 0
        reported_nav = nav_log.nav_per_unit

        if abs(expected_nav - reported_nav) > 0.01:
            breaks.append({
                "rule_id": "NAV_VAL_002",
                "break_type": "NAV_CALCULATION_ERROR",
                "calculated_nav": expected_nav,
                "reported_nav": reported_nav,
                "aum": nav_log.aum,
                "total_units": total_units,
                "variance": abs(expected_nav - reported_nav),
                "severity": SeverityLevel.CRITICAL,
                "message": f"NAV per unit calculation error: expected {expected_nav}, reported {reported_nav}"
            })

```

```

        return breaks

def _log_exceptions(self, breaks: list, rule_id: str, severity: SeverityLevel) -> None:
    """
    Log detected breaks to exceptions table.
    """
    for break_item in breaks:
        exception_record = {
            "tenant_id": self.tenant_id,
            "rule_id": rule_id,
            "severity": severity,
            "status": BreakStatus.OPEN,
            "detection_timestamp": datetime.now(),
            "break_details": break_item, # JSON
            "break_summary": break_item.get("message", ""),
            "domain": break_item.get("domain", self._infer_domain_from_rule(rule_id)),
            "affected_entities": {
                "trade_id": break_item.get("trade_id"),
                "cash_id": break_item.get("cash_id"),
            }
        }

        # Insert into exceptions table
        self.db.execute(text("""
            INSERT INTO exceptions (tenant_id, rule_id, severity, status, detection_timestamp,
                                   break_details, break_summary, domain, affected_entities)
            VALUES (:tenant_id, :rule_id, :severity, :status, :detection_timestamp,
                    :break_details, :break_summary, :domain, :affected_entities)
            """), exception_record)

        self.exceptions_created.append(exception_record)

    self.db.commit()

def _count_exceptions_by_severity(self) -> dict:
    """Count open exceptions by severity."""
    summary = {
        "CRITICAL": 0,
        "HIGH": 0,
        "MEDIUM": 0,
        "LOW": 0
    }

    for exc in self.exceptions_created:
        severity = exc["severity"]
        summary[severity] += 1

    return summary

def _escalate_exceptions_to_ops(self) -> None:
    """
    Push open exceptions to ops queue (Slack integration, dashboard notification).
    """
    critical_breaks = [e for e in self.exceptions_created if e["severity"] == SeverityLevel.CRITICAL]
    high_breaks = [e for e in self.exceptions_created if e["severity"] == SeverityLevel.HIGH]

```

```

    if critical_breaks:
        self._notify_slack(
            channel="#ops-critical",
            message=f"⚠️ {len(critical_breaks)} CRITICAL reconciliation breaks detected",
            breaks=critical_breaks[:5]  # First 5
        )

    if high_breaks:
        self._notify_slack(
            channel="#ops-alerts",
            message=f"⚠️ {len(high_breaks)} HIGH breaks detected",
            breaks=high_breaks[:3]
        )

def _generate_break_summary_report(self) -> None:
    """Generate and store summary report."""
    report = {
        "run_date": self.as_of_date,
        "tenant_id": self.tenant_id,
        "total_exceptions": len(self.exceptions_created),
        "by_severity": self._count_exceptions_by_severity(),
        "by_rule": self._count_exceptions_by_rule(),
        "execution_summary": self._build_summary_text()
    }

    self.db.execute(text("""
        INSERT INTO recon_reports (tenant_id, report_date, report_data)
        VALUES (:tenant_id, :report_date, :report_data)
    """), {
        "tenant_id": self.tenant_id,
        "report_date": self.as_of_date,
        "report_data": json.dumps(report)
    })
    self.db.commit()

# ===== HELPER METHODS =====

def _load_settled_trades(self, as_of_date: datetime.date) -> list:
    """Load all trades settled on or before as_of_date."""
    result = self.db.execute(text("""
        SELECT * FROM broker_trades
        WHERE tenant_id = :tenant_id AND settlement_date <= :as_of_date
        ORDER BY settlement_date DESC
    """), {"tenant_id": self.tenant_id, "as_of_date": as_of_date})
    return [dict(row) for row in result]

def _load_bank_txns(self, as_of_date: datetime.date) -> list:
    """Load bank txns."""
    result = self.db.execute(text("""
        SELECT * FROM bank_txns
        WHERE tenant_id = :tenant_id AND value_date <= :as_of_date
        ORDER BY value_date DESC
    """), {"tenant_id": self.tenant_id, "as_of_date": as_of_date})
    return [dict(row) for row in result]

def _load_holdings(self, as_of_date: datetime.date) -> list:

```

```

        """Load holdings as of date."""
        result = self.db.execute(text("""
            SELECT * FROM holdings
            WHERE tenant_id = :tenant_id AND as_of_date = :as_of_date
        """), {"tenant_id": self.tenant_id, "as_of_date": as_of_date})
        return [dict(row) for row in result]

def _get_cash_balance(self, as_of_date: datetime.date) -> float:
    """Get total cash balance (sum of all bank accounts)."""
    result = self.db.execute(text("""
        SELECT COALESCE(SUM(running_balance), 0) as total_cash
        FROM bank_txns
        WHERE tenant_id = :tenant_id AND value_date <= :as_of_date
        ORDER BY value_date DESC
        LIMIT 1
    """), {"tenant_id": self.tenant_id, "as_of_date": as_of_date})
    row = result.fetchone()
    return float(row[0]) if row else 0.0

def _get_total_units(self) -> float:
    """Get fund's total units outstanding."""
    result = self.db.execute(text("""
        SELECT total_units FROM fund_config
        WHERE tenant_id = :tenant_id
        LIMIT 1
    """), {"tenant_id": self.tenant_id})
    row = result.fetchone()
    return float(row[0]) if row else 0.0

def _notify_slack(self, channel: str, message: str, breaks: list) -> None:
    """Send Slack notification (stub)."""
    # Implementation: call Slack API with message + break details
    pass

def _infer_domain_from_rule(self, rule_id: str) -> str:
    """Infer domain from rule ID (e.g., TRADE_CASH_001 → Trade)."""
    if rule_id.startswith("TRADE_CASH"):
        return "Trade_Cash"
    elif rule_id.startswith("NAV_VAL"):
        return "NAV_Valuation"
    # ... etc
    return "Other"

def _count_exceptions_by_rule(self) -> dict:
    """Count exceptions by rule_id."""
    summary = {}
    for exc in self.exceptions_created:
        rule_id = exc["rule_id"]
        summary[rule_id] = summary.get(rule_id, 0) + 1
    return summary

def _build_summary_text(self) -> str:
    """Build human-readable summary."""
    return f"Reconciliation run for {self.tenant_id} on {self.as_of_date}: {len(self.

# ===== FASTAPI ENDPOINT =====

```

```

router = APIRouter(prefix="/api/reconciliation", tags=["reconciliation"])

@router.post("/run_daily/{tenant_id}")
async def run_daily_reconciliation(
    tenant_id: str,
    as_of_date: datetime.date = Query(default_factory=datetime.today)
):
    """
    Trigger daily reconciliation for a tenant.
    """
    try:
        engine = Phase1ReconciliationEngine(db_session, tenant_id, as_of_date)
        result = engine.run_phase1_daily_reconciliation()
        return {
            "status": "success",
            "result": result
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/exceptions/{tenant_id}")
async def get_open_exceptions(tenant_id: str):
    """
    Fetch open exceptions (break queue) for ops dashboard.
    """
    result = db_session.execute(text("""
        SELECT id, rule_id, severity, detection_timestamp, break_summary
        FROM exceptions
        WHERE tenant_id = :tenant_id AND status = 'OPEN'
        ORDER BY severity DESC, detection_timestamp DESC
    """)), {"tenant_id": tenant_id})

    exceptions = [dict(row) for row in result]
    return {
        "count": len(exceptions),
        "exceptions": exceptions
    }

@router.post("/exceptions/{exception_id}/resolve")
async def resolve_exception(
    exception_id: str,
    resolution_type: str, # CONFIRMED, CREATED_ENTRY, WAIVED, etc.
    notes: str,
    resolved_by_user_id: str
):
    """
    Ops manually resolves an exception.
    """
    # Update exception record
    db_session.execute(text("""
        UPDATE exceptions
        SET status = 'RESOLVED', resolution_type = :resolution_type,
            resolution_notes = :notes, resolved_by_user_id = :resolved_by_user_id,
            resolution_timestamp = NOW()
        WHERE id = :exception_id
    """))

```

```

        """), {
            "exception_id": exception_id,
            "resolution_type": resolution_type,
            "notes": notes,
            "resolved_by_user_id": resolved_by_user_id
        })

# Log to learning_events for self-healing
db_session.execute(text("""
    INSERT INTO learning_events (exception_id, resolution_type, notes, resolved_by_us
    VALUES (:exception_id, :resolution_type, :notes, :resolved_by_user_id, NOW())
"""), {
    "exception_id": exception_id,
    "resolution_type": resolution_type,
    "notes": notes,
    "resolved_by_user_id": resolved_by_user_id
})

db_session.commit()

return {"status": "resolved", "exception_id": exception_id}

```

7.1.5 Testing Strategy (Phase 1)

Unit Tests (Week 2-3):

- └─ Test exact match logic (TRADE_CASH_001)
 - └─ Test case: matched trade+cash (quantity 100, amount INR 10,000)
 - └─ Test case: no matching cash
 - └─ Test case: multiple matching cash entries
 - └─ Test case: amount off by 1 rupee (tolerance check)
- └─ Test fee decomposition (TRADE_CASH_003)
 - └─ Test STT calc for SELL (0.1%)
 - └─ Test STT calc for BUY (should be 0)
 - └─ Test GST on fees (18%)
 - └─ Test edge case: very small trade (rounding errors)
- └─ Test NAV calcs (NAV_VAL_001, NAV_VAL_002)
 - └─ Test AUM = sum(holdings) + cash
 - └─ Test NAV per unit = AUM / units
 - └─ Test with multi-currency holdings

Integration Tests (Week 4):

- └─ End-to-end flow with real broker/bank files
 - └─ Load sample broker trade file (XLSX)
 - └─ Load sample bank statement (CSV or PDF)
 - └─ Run full reconciliation
 - └─ Verify matched count is correct
 - └─ Verify breaks are detected
 - └─ Verify exception records created in DB
- └─ Ops workflow testing
 - └─ Generate break → escalate → ops resolves → mark as RESOLVED
 - └─ Verify learning_events logged

Regression Tests (ongoing):

- └ Run on 30 days of historical data (if available)
- └ Measure % auto-reconciled (target: 85%)
- └ Identify any systematic patterns (repeat breaks)

7.1.6 Deployment Checklist

Pre-Deployment (Day before):

- ☐ All unit tests passing
- ☐ Integration tests on staging (real data)
- ☐ Ops team trained on new workflows (manual fix form, escalation queue)
- ☐ Slack integration configured (#ops-critical, #ops-alerts)
- ☐ Database schema deployed (exceptions, learning_events tables)
- ☐ Config: tenant-specific tolerances loaded from rule_definitions
- ☐ Documentation: ops runbook written and reviewed

Deployment (Production):

- ☐ Deploy rule_engine code to Render
- ☐ Enable reconciliation cron job (daily, 8 PM IST)
- ☐ Monitor: logs, exception queue, performance metrics
- ☐ Have on-call ops person available Day 1 (to handle unexpected breaks)

Day 1 Post-Deployment:

- ☐ Run manual reconciliation on live data (confirm % auto-resolved)
- ☐ Review exceptions: are they reasonable?
- ☐ Iterate: adjust tolerances if needed (e.g., if false positives >5%)
- ☐ Gather ops feedback: are manual fixes intuitive?

Days 2-7:

- ☐ Daily standup on exception trends
- ☐ Analyze learning_events for patterns
- ☐ Fine-tune thresholds based on data
- ☐ Prepare Phase 2 roadmap updates (if any)

7.2 PHASE 2: POSITIONS + CORPORATE ACTIONS (WEEKS 5-8)

Objective: Extend reconciliation to **holdings** ↔ **custodian** and **corporate actions**, bringing coverage to 95%.

New Rules to Implement:

- POS_CUST_001 (holdings qty match)
- POS_CUST_002 (CA adjustments)
- CA_001-CA_012 (all CA types: splits, bonus, dividend, rights, mergers)
- SUS_002-SUS_005 (unmatched securities, partial fills, late settlement, etc.)

Key Deliverables:

Week 5:

- | Design: CA processing logic (ex-date, record-date, adjustment timeline)
- | Implement: POS_CUST_001 (holdings qty match) + unit tests
- | Implement: CA_001 (split adjustment) + CA_002 (bonus) + tests
- | Database: add corp_actions processing table (tracks ex-date, record-date, payment-d

Week 6:

- | Implement: CA_004 (dividend cash reconciliation)
- | Implement: CA_012 (reverse split)
- | Implement: POS_CUST_002 (CA impact on holdings)
- | Integrate: CA logic into daily recon flow (process CAs before holdings match)

Week 7:

- | Implement: SUS_002 (unmatched security detection)
- | Implement: SUS_004 (late settlement flagging)
- | Implement: TRADE_CASH_002 (partial fill handling)
- | Test: CA scenarios with real data (splits, dividends, rights)

Week 8:

- | QA: full integration (trades → cash → holdings → NAV with CAs)
- | Staging: run on 30-60 days of data
- | Update ops runbook: CA workflows, dividend payment tracking
- | Soft launch: deploy to staging; monitor 1 week

Success Metrics:

- % of trades auto-reconciled: ≥90% (up from 85%)
- % of holdings auto-reconciled: ≥80% (new)
- CA-related breaks: <2% false positives
- NAV variance: <0.5% for well-formed portfolios

7.3 PHASE 3: AI/FUZZY LAYER + SELF-HEALING (WEEKS 9-12)

Objective: Deploy Tier-3 AI rules + self-healing engine. Reduce ops load by 30-40%.

Key Components:

Week 9:

- | Design: AI rule architecture (when to call LLM, prompts, response parsing)
- | Implement: HYBRID rules (e.g., TRADE_CASH_002 partial fill with AI candidate scorin
- | Integrate: call_llm() gateway (via llm_gateway.py)
- | Implement: Fuzzy string matching for broker narratives (e.g., "Goldman Sachs" vs "G

Week 10:

- | Implement: Self-healing engine (batch analysis of learning_events)
- | Implement: Pattern detection (LLM analyzes 30-day learning data)
- | Implement: Rule update proposal (LLM suggests new/modified rules)
- | Implement: Tolerance tuning (auto-adjust thresholds based on patterns)
- | Test: self-healing on Phase 1 data

Week 11:

- └ Implement: Exception context enrichment (pull related data for AI reasoning)
- └ Implement: Multi-step AI reasoning (e.g., "is this timing issue or missing trade?")
- └ Integrate: AI layer into daily recon flow (after Tier-1/2, before ops escalation)
- └ Test: AI-driven auto-resolution on residual breaks

Week 12:

- └ QA: end-to-end with AI (live data, real LLM calls)
- └ Performance tuning: optimize LLM call batching, response parsing
- └ Deploy to production: full 3-tier engine
- └ Ops training: when to trust AI, how to override, when to escalate
- └ Success metrics: % auto-resolved (target: 95%+), ops time saved (target: 6-8 hours/

Expected Outcomes (by end of Week 12):

- 95%+ of breaks auto-resolved or auto-closed (Tier-1 + 2 + 3)
- <5% requiring manual ops intervention
- ~8-10 hours/day time savings for ops team
- Self-healing engine tuning tolerances nightly (0-2% impact on future false positives)

7.4 Phase Summary Table

Phase	Duration	Scope	Rules	Coverage	Success Metrics
Phase 1 (MVP)	4 weeks	Trade ↔ Cash, Basic NAV	10 rules (Tier-1/2 only)	~90% of breaks	85% auto-reconciled, <4 hrs ops per break
Phase 2	4 weeks	Holdings, CA, Suspense	+15 rules (POS, CA, SUS)	~97% of breaks	90% auto-reconciled, 2-3 hrs ops per break
Phase 3	4 weeks	AI, Self-Healing	+20 rules (AI/hybrid)	99%+ of breaks	95%+ auto-resolved, <1 hr ops per residual break

SECTION 8 — DATA MODEL & CTM REFINEMENTS

8.1 Current State (Your Existing Schema)

You already have normalized tables:

```
-- Already exist in your Render PostgreSQL:
broker_trades(trade_id, trade_date, settlement_date, symbol, isin, quantity,
              price, gross_amount, net_amount, side, currency, broker, status, ...)

bank_txns(txn_id, value_date, booking_date, description, amount, running_balance,
          currency, status, counterparty, source_system, ...)

holdings(position_id, as_of_date, symbol, isin, quantity, cost_base, market_price,
          total_value, currency, source, ...)

nav_logs(nav_date, fund_isin, nav_per_unit, total_aum, total_units, currency, source, ...)
```

```
corp_actions(ca_id, ex_date, record_date, symbol, isin, type, ratio_numerator,
             ratio_denominator, cash_amount, currency, ...)
```

8.2 Proposed Enhancements (for Reconciliation Engine)

Add these helper tables + fields to your schema:

A. Exceptions Table (Core for break management)

```
CREATE TABLE exceptions (
  id BIGSERIAL PRIMARY KEY,
  tenant_id UUID NOT NULL,
  rule_id VARCHAR(50) NOT NULL,           -- e.g., TRADE_CASH_001
  domain VARCHAR(30) NOT NULL,           -- Trade_Cash | Holdings | NAV | FX | CA |
  severity VARCHAR(20) NOT NULL,         -- CRITICAL | HIGH | MEDIUM | LOW
  status VARCHAR(30) NOT NULL,           -- OPEN | ESCALATED_TO_OPS | IN_INVESTIGATION

  -- Break details (JSON, for flexibility)
  break_type VARCHAR(50),                 -- e.g., NO_MATCHING_CASH, QTY_MISMATCH, T
  break_details JSONB,                   -- Detailed context: { trade_id, cash_id,
  break_summary TEXT,                    -- Human-readable 1-liner

  -- Affected entities (for querying)
  affected_trade_id BIGINT,               -- Link to broker_trades.id (nullable)
  affected_cash_id BIGINT,               -- Link to bank_txns.id (nullable)
  affected_position_id BIGINT,           -- Link to holdings.id (nullable)
  affected_nav_id BIGINT,                -- Link to nav_logs.id (nullable)

  -- Lifecycle tracking
  detection_timestamp TIMESTAMP NOT NULL DEFAULT NOW(),
  escalation_timestamp TIMESTAMP,
  investigation_notes TEXT,
  resolution_type VARCHAR(50),           -- CONFIRMED | CREATED_ENTRY | WAIVED | O
  resolution_notes TEXT,
  resolved_by_user_id UUID,              -- ops team member who resolved
  resolution_timestamp TIMESTAMP,
  approval_user_id UUID,                 -- PM/Compliance approval (if required)
  approval_timestamp TIMESTAMP,

  -- Aging + SLA tracking
  sla_minutes INT,                       -- Expected resolution time (per severity)
  minutes_open INT GENERATED ALWAYS AS
    (EXTRACT(EPOCH FROM (COALESCE(resolution_timestamp, NOW()) - detection_timestamp)
    STORED,
  sla_breached BOOLEAN GENERATED ALWAYS AS (minutes_open > sla_minutes) STORED,

  -- Audit trail
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),

  CONSTRAINT fk_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id),
  CONSTRAINT fk_trade FOREIGN KEY (affected_trade_id) REFERENCES broker_trades(id) ON DELETE
  CONSTRAINT fk_cash FOREIGN KEY (affected_cash_id) REFERENCES bank_txns(id) ON DELETE
```

```

CONSTRAINT fk_position FOREIGN KEY (affected_position_id) REFERENCES holdings(id) ON

INDEX idx_tenant_status (tenant_id, status),
INDEX idx_tenant_domain_severity (tenant_id, domain, severity),
INDEX idx_detection_timestamp (detection_timestamp DESC),
INDEX idx_affected_trade (affected_trade_id),
INDEX idx_affected_cash (affected_cash_id)
);

```

B. Learning Events Table (For self-healing)

```

CREATE TABLE learning_events (
  id BIGSERIAL PRIMARY KEY,
  tenant_id UUID NOT NULL,
  exception_id BIGINT NOT NULL,
  rule_id VARCHAR(50) NOT NULL,
  domain VARCHAR(30) NOT NULL,

  -- What was the resolution?
  resolution_type VARCHAR(50),           -- CONFIRMED | CREATED_ENTRY | FEE_ADJUSTMENT
  fields_adjusted JSONB,                 -- { "field_name": { old_value, new_value } }
  resolution_notes TEXT,

  -- Who resolved it?
  resolved_by_user_id UUID NOT NULL,
  resolution_date TIMESTAMP NOT NULL,

  -- Pattern detection markers
  is_pattern BOOLEAN DEFAULT FALSE,      -- Flag if this resolves a repeated issue
  pattern_signature VARCHAR(255),        -- Hash of pattern (for grouping similar issues)

  created_at TIMESTAMP DEFAULT NOW(),

  CONSTRAINT fk_exception FOREIGN KEY (exception_id) REFERENCES exceptions(id) ON DELETE CASCADE,
  CONSTRAINT fk_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id),

  INDEX idx_tenant_rule (tenant_id, rule_id),
  INDEX idx_domain_pattern (domain, is_pattern),
  INDEX idx_pattern_signature (pattern_signature)
);

```

C. Rule Definitions Table (Configuration + versioning)

```

CREATE TABLE rule_definitions (
  id SERIAL PRIMARY KEY,
  tenant_id UUID,                       -- NULL = global rule; non-NULL = tenant specific
  rule_id VARCHAR(50) NOT NULL,         -- TRADE_CASH_001, etc.
  domain VARCHAR(30) NOT NULL,
  rule_name VARCHAR(255) NOT NULL,
  rule_description TEXT,

```

```

-- Rule logic
rule_type VARCHAR(30),                -- DETERMINISTIC | TOLERANCE | DATA_QUALITY
tier INT,                            -- 1 | 2 | 3 | 4 (Tier-4 = AI)
should_live_in VARCHAR(30),          -- RULE_ENGINE | AI_LAYER | HYBRID

-- Tolerances (parameterized per tenant/region)
tolerance_abs DECIMAL(15,2),         -- e.g., 1.00 rupee
tolerance_pct DECIMAL(5,2),          -- e.g., 0.10 %
tolerance_days INT,                  -- e.g., 2 days
tolerance_qty INT,

-- Applicability
region_code VARCHAR(10),              -- IN | US | EU | GLOBAL
is_active BOOLEAN DEFAULT TRUE,
effective_from_date DATE,
effective_to_date DATE,

-- Metadata
severity_level VARCHAR(20),           -- CRITICAL | HIGH | MEDIUM | LOW
created_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW(),
updated_by_user_id UUID,              -- Who last updated? (for audit)
update_reason TEXT,                  -- Why was tolerance changed?

-- Version control
rule_version INT DEFAULT 1,           -- Incremented on each change
previous_tolerance_abs DECIMAL(15,2), -- Track history for self-healing analysis
previous_tolerance_pct DECIMAL(5,2),

CONSTRAINT unique_rule_tenant_effective
    UNIQUE (rule_id, tenant_id, effective_from_date),

INDEX idx_tenant_active (tenant_id, is_active),
INDEX idx_rule_id (rule_id),
INDEX idx_effective_dates (effective_from_date, effective_to_date)
);

```

D. Reconciliation Runs / Reports Table

```

CREATE TABLE recon_runs (
    id BIGSERIAL PRIMARY KEY,
    tenant_id UUID NOT NULL,
    run_date DATE NOT NULL,
    run_timestamp TIMESTAMP NOT NULL DEFAULT NOW(),

    -- Run summary
    total_trades INT,
    matched_trades INT,
    unmatched_trades INT,
    total_exceptions INT,
    exceptions_by_severity JSONB,      -- { CRITICAL: N, HIGH: N, ... }
    exceptions_by_domain JSONB,
    exceptions_by_rule JSONB,

```

```

-- Performance metrics
execution_time_seconds DECIMAL(10,2),
rules_executed INT,
rules_passed INT,
rules_failed INT,

-- Status
status VARCHAR(30), -- COMPLETED | FAILED | PARTIAL
error_message TEXT,

-- Phase info
phase VARCHAR(20), -- PHASE_1 | PHASE_2 | PHASE_3
tier_distribution JSONB, -- { TIER_1: N_rules, TIER_2: N, TIER_3: N }

created_at TIMESTAMP DEFAULT NOW(),

CONSTRAINT fk_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id),
INDEX idx_tenant_date (tenant_id, run_date DESC),
INDEX idx_run_status (status)
);

```

E. Broker Quirks / Exceptions Registry (For AI learning)

```

CREATE TABLE broker_quirks (
  id SERIAL PRIMARY KEY,
  tenant_id UUID, -- NULL = global; non-NULL = tenant-specific
  broker_name VARCHAR(100) NOT NULL,

  -- Known quirks
  quirk_type VARCHAR(50), -- FEE_ROUNDING | TIMING_VARIANCE | SYMBOL_MISMATCH
  quirk_description TEXT,

  -- How to handle it
  handling_rule_id VARCHAR(50), -- Link to rule_definitions.rule_id
  tolerance_override JSONB, -- { tolerance_abs: X, tolerance_pct: Y }

  -- Confidence & tracking
  confidence_score DECIMAL(3,2), -- 0.0-1.0 (how confident are we about this)
  occurrences_count INT, -- How many times observed?
  last_observed_date DATE,

  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW(),

  INDEX idx_broker_name (broker_name),
  INDEX idx_quirk_type (quirk_type)
);

```

F. FX Rates Table (For multi-currency reconciliation)

```
CREATE TABLE fx_rates (  
    id BIGSERIAL PRIMARY KEY,  
    tenant_id UUID NOT NULL,  
    rate_date DATE NOT NULL,  
    rate_time TIMESTAMP,                -- For intra-day if needed  
  
    from_currency VARCHAR(3) NOT NULL,  -- e.g., USD  
    to_currency VARCHAR(3) NOT NULL,    -- e.g., INR  
    rate_value DECIMAL(20,8) NOT NULL,  -- e.g., 83.50  
  
    -- Source metadata  
    rate_source VARCHAR(50),            -- CUSTODIAN | MARKET_FEED | DEAL | MANUAL  
    is_deal_rate BOOLEAN,               -- Is this a locked deal rate?  
  
    -- Validity  
    is_official BOOLEAN,                -- Is this custodian's official rate for M  
    effective_from_time TIMESTAMP,  
    effective_to_time TIMESTAMP,  
  
    created_at TIMESTAMP DEFAULT NOW(),  
  
    CONSTRAINT fk_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id),  
    CONSTRAINT unique_rate UNIQUE (tenant_id, rate_date, from_currency, to_currency, rate_value),  
  
    INDEX idx_tenant_date (tenant_id, rate_date DESC),  
    INDEX idx_rate_source (rate_source)  
);
```

8.3 Canonical Transaction Model (CTM) — Python Enhancements

Your existing CTM (in `backend/rule_engine/core/ctm.py`) should be extended:

```
# File: backend/rule_engine/core/ctm.py  
  
from dataclasses import dataclass, field  
from typing import Optional, Dict, List  
from datetime import datetime, date  
from decimal import Decimal  
from enum import Enum  
  
class TradeStatus(str, Enum):  
    UNSETTLED = "UNSETTLED"  
    SETTLED = "SETTLED"  
    CANCELLED = "CANCELLED"  
    REJECTED = "REJECTED"  
    PARTIALLY_FILLED = "PARTIALLY_FILLED"  
  
class TradeSide(str, Enum):  
    BUY = "BUY"  
    SELL = "SELL"
```

```

class CashStatus(str, Enum):
    PENDING = "PENDING"
    POSTED = "POSTED"
    REVERSED = "REVERSED"
    FAILED = "FAILED"
    CONFIRMED = "CONFIRMED"

@dataclass
class Trade:
    """Core trade object."""
    id: str
    tenant_id: str
    broker: str
    trade_date: date
    settlement_date: date

    # Security identifiers
    symbol: str
    isin: str
    sedol: Optional[str] = None
    cusip: Optional[str] = None

    # Trade specifics
    side: TradeSide
    quantity: int
    price: Decimal
    gross_amount: Decimal

    # Fees (broken down)
    brokerage: Optional[Decimal] = Decimal('0')
    stt: Optional[Decimal] = Decimal('0') # India-specific
    exchange_fee: Optional[Decimal] = Decimal('0')
    gst: Optional[Decimal] = Decimal('0') # India-specific
    other_fees: Optional[Decimal] = Decimal('0')

    net_amount: Decimal

    # Currencies
    trade_currency: str # e.g., USD
    settlement_currency: str # e.g., INR (may differ if FX forward)
    base_currency: str = 'INR'

    # FX (if multi-currency)
    fx_forward_id: Optional[str] = None
    fx_rate_used: Optional[Decimal] = None
    fx_rate_source: Optional[str] = None # CUSTODIAN | MARKET | DEAL

    # Status & reconciliation
    status: TradeStatus = TradeStatus.UNSETTLED
    matched_cash_id: Optional[str] = None
    matched_cash_date: Optional[date] = None

    # Metadata
    source_file: Optional[str] = None
    broker_reference: Optional[str] = None
    trade_id_at_broker: Optional[str] = None

```

```

        created_at: datetime = field(default_factory=datetime.now)
        updated_at: datetime = field(default_factory=datetime.now)

@dataclass
class CashTxn:
    """Bank/custodian cash movement."""
    id: str
    tenant_id: str
    value_date: date
    booking_date: date

    description: str
    amount: Decimal
    currency: str
    running_balance: Decimal

    status: CashStatus = CashStatus.PENDING
    counterparty: Optional[str] = None
    source_system: str = 'BANK' # BANK | CUSTODIAN | BROKER

    # Reconciliation
    matched_trade_id: Optional[str] = None
    matched_trade_date: Optional[date] = None

    # For dividend / CA cash
    related_corp_action_id: Optional[str] = None

    created_at: datetime = field(default_factory=datetime.now)

@dataclass
class Position:
    """Holdings / positions."""
    id: str
    tenant_id: str
    as_of_date: date

    # Security
    symbol: str
    isin: str

    # Quantity & valuation
    quantity: int
    cost_per_share: Optional[Decimal] = None
    cost_base: Optional[Decimal] = None # Total cost (qty × cost_per_share)
    market_price: Decimal
    total_value: Decimal

    currency: str = 'INR'
    price_source: Optional[str] = None # CUSTODIAN | MARKET_FEED | MANUAL
    price_as_of_date: Optional[date] = None

    # Custodian vs Internal
    source: str = 'INTERNAL' # INTERNAL | CUSTODIAN

    # Corporate action adjustments

```

[illegible]

```

# Dates
announcement_date: Optional[date] = None
ex_date: date
record_date: date
payment_date: date

# Ratio (for splits/bonus)
ratio_numerator: Optional[int] = None
ratio_denominator: Optional[int] = None

# Cash impact (for dividends)
dividend_per_share: Optional[Decimal] = None
dividend_total_cash: Optional[Decimal] = None
currency: str = 'INR'

# Metadata
source: str = 'PROVIDER'                                # PROVIDER | MANUAL | EXCHANGE
confirmed: bool = False

# Processing status
processed: bool = False
processed_date: Optional[date] = None

created_at: datetime = field(default_factory=datetime.now)

@dataclass
class Exception:
    """Reconciliation break / exception."""
    id: str
    tenant_id: str
    rule_id: str
    domain: str
    severity: str                                         # CRITICAL | HIGH | MEDIUM | LOW
    status: str                                           # OPEN | ESCALATED | RESOLVED | CLOSED

    break_type: str
    break_details: Dict
    break_summary: str

    affected_trade_id: Optional[str] = None
    affected_cash_id: Optional[str] = None
    affected_position_id: Optional[str] = None

    detection_timestamp: datetime = field(default_factory=datetime.now)
    resolution_timestamp: Optional[datetime] = None
    resolved_by_user_id: Optional[str] = None

    created_at: datetime = field(default_factory=datetime.now)

```

8.4 Database Migration Script (for Phase 1 Deployment)

```
-- File: backend/migrations/001_reconciliation_schema.sql

-- 1. Exceptions table
CREATE TABLE IF NOT EXISTS exceptions (
    id BIGSERIAL PRIMARY KEY,
    tenant_id UUID NOT NULL,
    rule_id VARCHAR(50) NOT NULL,
    domain VARCHAR(30) NOT NULL,
    severity VARCHAR(20) NOT NULL CHECK (severity IN ('CRITICAL', 'HIGH', 'MEDIUM', 'LOW')),
    status VARCHAR(30) NOT NULL DEFAULT 'OPEN' CHECK (status IN ('OPEN', 'ESCALATED_TO_OPEN')),

    break_type VARCHAR(50),
    break_details JSONB,
    break_summary TEXT,

    affected_trade_id BIGINT REFERENCES broker_trades(id) ON DELETE CASCADE,
    affected_cash_id BIGINT REFERENCES bank_txns(id) ON DELETE CASCADE,
    affected_position_id BIGINT REFERENCES holdings(id) ON DELETE CASCADE,

    detection_timestamp TIMESTAMP NOT NULL DEFAULT NOW(),
    escalation_timestamp TIMESTAMP,
    investigation_notes TEXT,
    resolution_type VARCHAR(50),
    resolution_notes TEXT,
    resolved_by_user_id UUID,
    resolution_timestamp TIMESTAMP,
    approval_user_id UUID,
    approval_timestamp TIMESTAMP,

    sla_minutes INT,
    minutes_open INT GENERATED ALWAYS AS (EXTRACT(EPOCH FROM (COALESCE(resolution_timestamp, NOW()) - detection_timestamp))) STORED,
    sla_breached BOOLEAN GENERATED ALWAYS AS (minutes_open > sla_minutes) STORED,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),

    CONSTRAINT fk_tenant_exceptions FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE
);

CREATE INDEX idx_exceptions_tenant_status ON exceptions(tenant_id, status);
CREATE INDEX idx_exceptions_severity ON exceptions(tenant_id, domain, severity);
CREATE INDEX idx_exceptions_timestamp ON exceptions(detection_timestamp DESC);
CREATE INDEX idx_exceptions_trade ON exceptions(affected_trade_id);
CREATE INDEX idx_exceptions_cash ON exceptions(affected_cash_id);

-- 2. Learning events table
CREATE TABLE IF NOT EXISTS learning_events (
    id BIGSERIAL PRIMARY KEY,
    tenant_id UUID NOT NULL,
    exception_id BIGINT NOT NULL REFERENCES exceptions(id) ON DELETE CASCADE,
    rule_id VARCHAR(50) NOT NULL,
    domain VARCHAR(30) NOT NULL,

    resolution_type VARCHAR(50),
```

```

fields_adjusted JSONB,
resolution_notes TEXT,

resolved_by_user_id UUID NOT NULL,
resolution_date TIMESTAMP NOT NULL,

is_pattern BOOLEAN DEFAULT FALSE,
pattern_signature VARCHAR(255),

created_at TIMESTAMP DEFAULT NOW(),

CONSTRAINT fk_tenant_learning FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE
);

CREATE INDEX idx_learning_tenant_rule ON learning_events(tenant_id, rule_id);
CREATE INDEX idx_learning_domain_pattern ON learning_events(domain, is_pattern);
CREATE INDEX idx_learning_pattern_sig ON learning_events(pattern_signature);

-- 3. Rule definitions table
CREATE TABLE IF NOT EXISTS rule_definitions (
    id SERIAL PRIMARY KEY,
    tenant_id UUID,
    rule_id VARCHAR(50) NOT NULL,
    domain VARCHAR(30) NOT NULL,
    rule_name VARCHAR(255) NOT NULL,
    rule_description TEXT,

    rule_type VARCHAR(30),
    tier INT,
    should_live_in VARCHAR(30),

    tolerance_abs DECIMAL(15,2),
    tolerance_pct DECIMAL(5,2),
    tolerance_days INT,
    tolerance_qty INT,

    region_code VARCHAR(10) DEFAULT 'GLOBAL',
    is_active BOOLEAN DEFAULT TRUE,
    effective_from_date DATE,
    effective_to_date DATE,

    severity_level VARCHAR(20),

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),
    updated_by_user_id UUID,
    update_reason TEXT,

    rule_version INT DEFAULT 1,
    previous_tolerance_abs DECIMAL(15,2),
    previous_tolerance_pct DECIMAL(5,2),

    CONSTRAINT fk_tenant_rules FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE
    CONSTRAINT unique_rule_tenant_eff UNIQUE (rule_id, tenant_id, effective_from_date)
);

```

```

CREATE INDEX idx_rules_tenant_active ON rule_definitions(tenant_id, is_active);
CREATE INDEX idx_rules_rule_id ON rule_definitions(rule_id);
CREATE INDEX idx_rules_effective ON rule_definitions(effective_from_date, effective_to_date);

-- 4. Reconciliation runs table
CREATE TABLE IF NOT EXISTS recon_runs (
    id BIGSERIAL PRIMARY KEY,
    tenant_id UUID NOT NULL,
    run_date DATE NOT NULL,
    run_timestamp TIMESTAMP NOT NULL DEFAULT NOW(),

    total_trades INT,
    matched_trades INT,
    unmatched_trades INT,
    total_exceptions INT,
    exceptions_by_severity JSONB,
    exceptions_by_domain JSONB,
    exceptions_by_rule JSONB,

    execution_time_seconds DECIMAL(10,2),
    rules_executed INT,
    rules_passed INT,
    rules_failed INT,

    status VARCHAR(30) CHECK (status IN ('COMPLETED', 'FAILED', 'PARTIAL')),
    error_message TEXT,

    phase VARCHAR(20),
    tier_distribution JSONB,

    created_at TIMESTAMP DEFAULT NOW(),

    CONSTRAINT fk_tenant_runs FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE
);

CREATE INDEX idx_runs_tenant_date ON recon_runs(tenant_id, run_date DESC);
CREATE INDEX idx_runs_status ON recon_runs(status);

-- 5. Broker quirks table
CREATE TABLE IF NOT EXISTS broker_quirks (
    id SERIAL PRIMARY KEY,
    tenant_id UUID,
    broker_name VARCHAR(100) NOT NULL,

    quirk_type VARCHAR(50),
    quirk_description TEXT,

    handling_rule_id VARCHAR(50),
    tolerance_override JSONB,

    confidence_score DECIMAL(3,2),
    occurrences_count INT,
    last_observed_date DATE,

    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT NOW(),

```

```

        CONSTRAINT fk_tenant_quirks FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE
    );

CREATE INDEX idx_quirks_broker ON broker_quirks(broker_name);
CREATE INDEX idx_quirks_type ON broker_quirks(quirk_type);

-- 6. FX rates table
CREATE TABLE IF NOT EXISTS fx_rates (
    id BIGSERIAL PRIMARY KEY,
    tenant_id UUID NOT NULL,
    rate_date DATE NOT NULL,
    rate_time TIMESTAMP,

    from_currency VARCHAR(3) NOT NULL,
    to_currency VARCHAR(3) NOT NULL,
    rate_value DECIMAL(20,8) NOT NULL,

    rate_source VARCHAR(50),
    is_deal_rate BOOLEAN,
    is_official BOOLEAN,
    effective_from_time TIMESTAMP,
    effective_to_time TIMESTAMP,

    created_at TIMESTAMP DEFAULT NOW(),

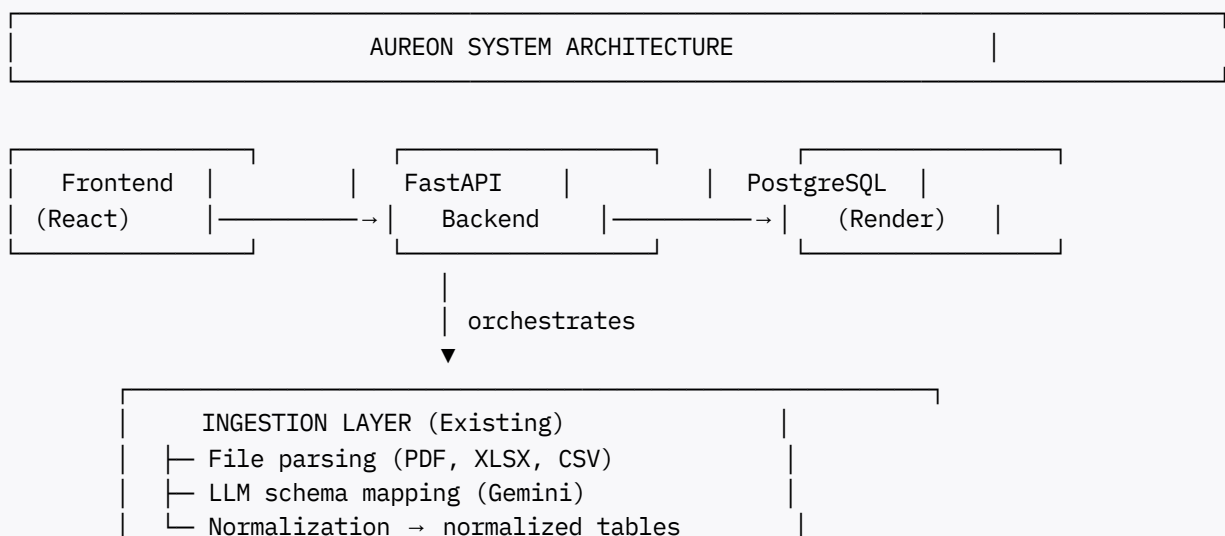
    CONSTRAINT fk_tenant_fxrates FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE
    CONSTRAINT unique_fx_rate UNIQUE (tenant_id, rate_date, from_currency, to_currency, 1
);

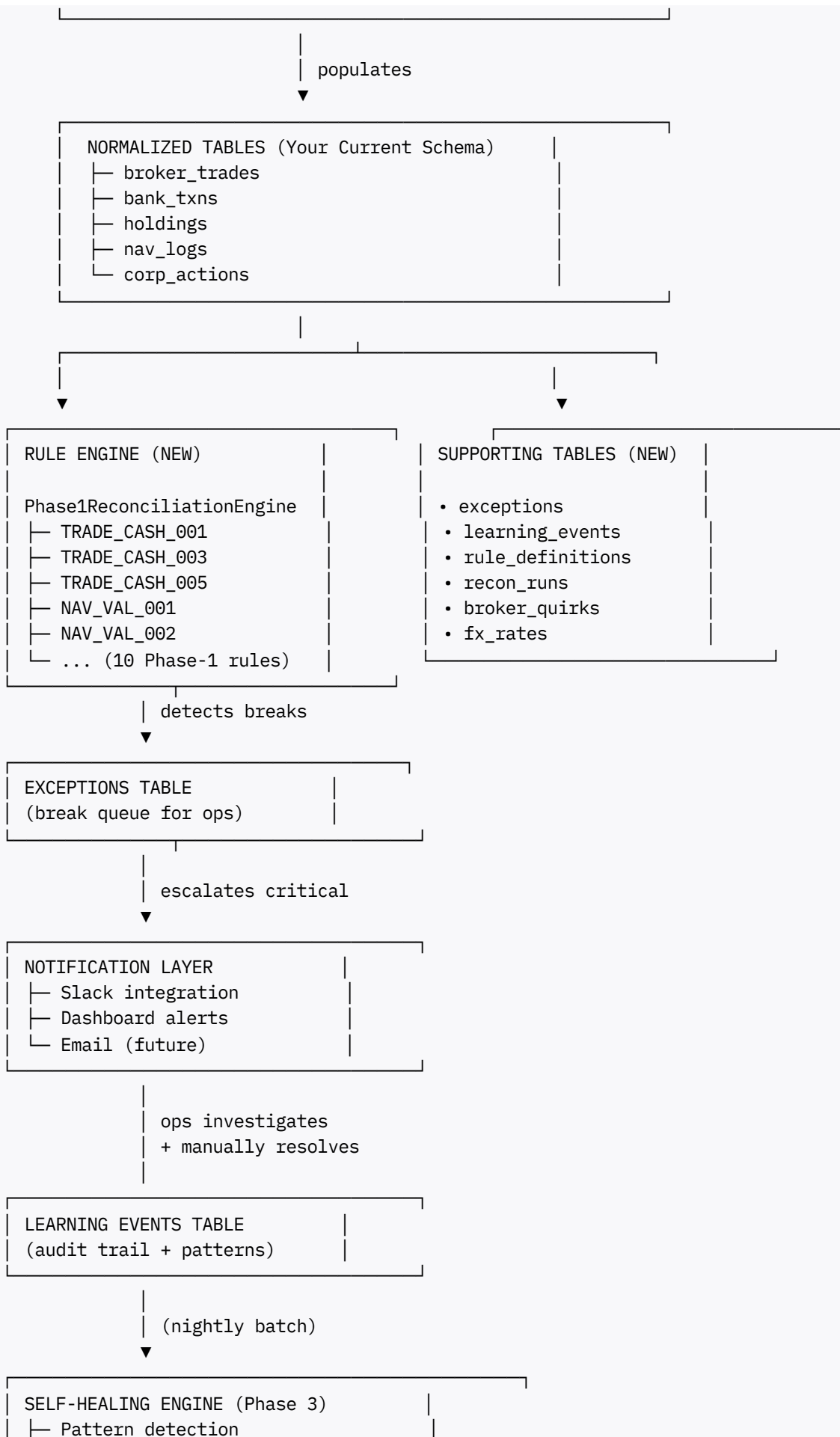
CREATE INDEX idx_fxrates_tenant_date ON fx_rates(tenant_id, rate_date DESC);
CREATE INDEX idx_fxrates_source ON fx_rates(rate_source);

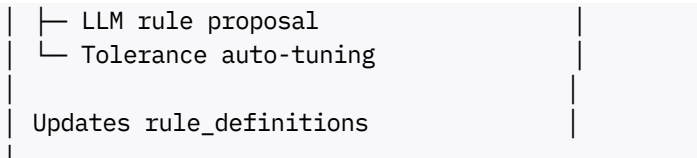
```

SECTION 9 — INTEGRATION ARCHITECTURE

9.1 System Architecture Diagram







9.2 Data Flow: Daily Batch Reconciliation

Time: 8:00 PM IST (end of business day)

- └ Cron job triggered: `run_daily_reconciliation(tenant_id, as_of_date=today)`
- └ Step 1: LOAD DATA
 - └ `broker_trades(settlement_date <= today) → memory`
 - └ `bank_txns(value_date <= today) → memory`
 - └ `holdings(as_of_date = today) → memory`
 - └ `nav_logs(nav_date = today) → memory`
 - └ `corp_actions(ex_date <= today) → memory`
- └ Step 2: VALIDATE DATA QUALITY (optional Tier-0)
 - └ Check: all required fields present
 - └ Check: no negative amounts on deposits
 - └ Check: dates in correct sequence
- └ Step 3: RUN TIER-1 DETERMINISTIC RULES (in order)
 - └ `TRADE_CASH_001`: exact match (80% of breaks resolved)
 - └ `TRADE_CASH_005`: status alignment
 - └ `TRADE_CASH_006`: timing check
 - └ `TRADE_CASH_008`: buy/sell sign
 - └ `NAV_VAL_001`: AUM sum
 - └ `NAV_VAL_002`: NAV per unit
 - └ `SUS_001`: unmatched cash detection
- └ Step 4: RUN TIER-2 TOLERANCE RULES (in order)
 - └ `TRADE_CASH_003`: fee decomposition
 - └ `NAV_VAL_003`: internal vs custodian NAV
 - └ ... (other tolerance rules)
- └ Step 5: CREATE EXCEPTIONS FOR ALL DETECTED BREAKS
 - └ For each break: INSERT into exceptions table
 - └ Set status = OPEN
 - └ Set severity (CRITICAL / HIGH / MEDIUM / LOW)
 - └ Set SLA (e.g., CRITICAL = 4 hours)
- └ Step 6: ESCALATE TO OPS
 - └ Query: exceptions where status = OPEN and severity in [CRITICAL, HIGH]
 - └ Send Slack notification to #ops-critical
 - " 5 CRITICAL breaks detected for [Fund Name]"
 - └ `TRADE_CASH_001`: 3 unsettled trades
 - └ `NAV_VAL_001`: 1 AUM variance
 - └ Push to ops dashboard (exceptions queue)
- └ Step 7: GENERATE REPORTS
 - └ Insert into `recon_runs`: summary statistics
 - └ Generate reconciliation report (JSON)

```

    {
        "run_date": "2025-12-05",
        "tenant_id": "...",
        "total_exceptions": 8,
        "by_severity": { CRITICAL: 5, HIGH: 2, MEDIUM: 1 },
        "by_domain": { Trade_Cash: 6, NAV: 2 },
        "by_rule": { TRADE_CASH_001: 3, ... }
    }
    └─ Store in recon_runs table

└─ Time: ~8:15 PM IST: Reconciliation complete
    Ops notified; can now investigate breaks

```

9.3 API Endpoints (Integration Points)

```

# File: backend/routes/reconciliation.py

from fastapi import APIRouter, HTTPException, Query, Depends
from sqlalchemy.orm import Session
from datetime import date, datetime
from typing import List, Optional
import json

router = APIRouter(prefix="/api/reconciliation", tags=["reconciliation"])

# ===== ORCHESTRATION ENDPOINTS =====

@router.post("/run_daily/{tenant_id}")
async def trigger_daily_reconciliation(
    tenant_id: str,
    as_of_date: date = Query(default_factory=date.today),
    db: Session = Depends(get_db)
):
    """
    Trigger daily reconciliation for a tenant.
    Input: tenant_id, optional as_of_date
    Output: summary (total exceptions, by severity)
    """
    try:
        engine = Phase1ReconciliationEngine(db, tenant_id, as_of_date)
        result = engine.run_phase1_daily_reconciliation()
        return {
            "status": "success",
            "result": result
        }
    except Exception as e:
        return {"status": "failed", "error": str(e)}

@router.post("/run_incremental/{tenant_id}")
async def trigger_incremental_reconciliation(
    tenant_id: str,
    file_upload_id: str,
    as_of_date: date = Query(default_factory=date.today),
    db: Session = Depends(get_db)
):

```

```

):
    """
    Incremental reconciliation when a new file is uploaded.
    Reprocesses only newly ingested entities.
    """
    # Load new entities from upload
    # Run subset of rules relevant to those entities
    # Return exceptions for new data only
    pass

# ===== EXCEPTION MANAGEMENT ENDPOINTS =====

@router.get("/exceptions/{tenant_id}")
async def get_exceptions(
    tenant_id: str,
    status: Optional[str] = Query(None), # OPEN, RESOLVED, etc.
    domain: Optional[str] = Query(None),
    severity: Optional[str] = Query(None),
    limit: int = Query(50),
    db: Session = Depends(get_db)
):
    """
    Fetch exceptions (break queue) for ops dashboard.
    Supports filtering by status, domain, severity.
    """
    query = db.query(Exception).filter(Exception.tenant_id == tenant_id)

    if status:
        query = query.filter(Exception.status == status)
    if domain:
        query = query.filter(Exception.domain == domain)
    if severity:
        query = query.filter(Exception.severity == severity)

    exceptions = query.order_by(Exception.severity.desc(), Exception.detection_timestamp).

    return {
        "count": len(exceptions),
        "exceptions": [
            {
                "id": e.id,
                "rule_id": e.rule_id,
                "domain": e.domain,
                "severity": e.severity,
                "status": e.status,
                "break_summary": e.break_summary,
                "detection_timestamp": e.detection_timestamp.isoformat(),
                "minutes_open": e.minutes_open,
                "sla_breached": e.sla_breached
            }
            for e in exceptions
        ]
    }

@router.get("/exceptions/{exception_id}")
async def get_exception_detail(

```

```

exception_id: str,
db: Session = Depends(get_db)
):
    """
    Fetch detailed context for an exception.
    Includes linked entities (trade, cash, position).
    """
    exc = db.query(Exception).filter(Exception.id == exception_id).first()

    if not exc:
        raise HTTPException(status_code=404, detail="Exception not found")

    # Fetch linked entities for context
    linked_data = {}
    if exc.affected_trade_id:
        trade = db.query(Trade).filter(Trade.id == exc.affected_trade_id).first()
        linked_data["trade"] = {
            "id": trade.id,
            "symbol": trade.symbol,
            "quantity": trade.quantity,
            "price": float(trade.price),
            "net_amount": float(trade.net_amount),
            "settlement_date": trade.settlement_date.isoformat(),
            "status": trade.status
        }

    if exc.affected_cash_id:
        cash = db.query(CashTxn).filter(CashTxn.id == exc.affected_cash_id).first()
        linked_data["cash"] = {
            "id": cash.id,
            "value_date": cash.value_date.isoformat(),
            "amount": float(cash.amount),
            "description": cash.description,
            "status": cash.status
        }

    return {
        "exception": {
            "id": exc.id,
            "rule_id": exc.rule_id,
            "domain": exc.domain,
            "severity": exc.severity,
            "status": exc.status,
            "break_type": exc.break_type,
            "break_details": exc.break_details,
            "break_summary": exc.break_summary,
            "detection_timestamp": exc.detection_timestamp.isoformat(),
            "minutes_open": exc.minutes_open,
            "sla_breached": exc.sla_breached
        },
        "linked_entities": linked_data
    }

@router.post("/exceptions/{exception_id}/resolve")
async def resolve_exception(
    exception_id: str,

```

```

resolution_type: str,          # CONFIRMED, CREATED_ENTRY, WAIVED, OVERRIDE, etc.
notes: str,
resolved_by_user_id: str,
approval_user_id: Optional[str] = None, # PM/Compliance approval (if needed)
db: Session = Depends(get_db)
):
    """
    Ops manually resolves an exception.
    Logs to learning_events for self-healing analysis.
    """
    exc = db.query(Exception).filter(Exception.id == exception_id).first()

    if not exc:
        raise HTTPException(status_code=404)

    # Update exception
    exc.status = "RESOLVED"
    exc.resolution_type = resolution_type
    exc.resolution_notes = notes
    exc.resolved_by_user_id = resolved_by_user_id
    exc.resolution_timestamp = datetime.now()

    if approval_user_id:
        exc.approval_user_id = approval_user_id
        exc.approval_timestamp = datetime.now()

    db.add(exc)
    db.commit()

    # Log to learning_events
    learning_event = LearningEvent(
        tenant_id=exc.tenant_id,
        exception_id=exc.id,
        rule_id=exc.rule_id,
        domain=exc.domain,
        resolution_type=resolution_type,
        fields_adjusted=json.dumps(exc.break_details),
        resolution_notes=notes,
        resolved_by_user_id=resolved_by_user_id,
        resolution_date=datetime.now()
    )
    db.add(learning_event)
    db.commit()

    return {
        "status": "resolved",
        "exception_id": exception_id,
        "resolution_type": resolution_type
    }

# ===== REPORTING ENDPOINTS =====

@router.get("/report/{tenant_id}/{run_date}")
async def get_recon_report(
    tenant_id: str,
    run_date: date,

```

```

db: Session = Depends(get_db)
):
    """
    Fetch reconciliation report for a specific date.
    """
    report = db.query(ReconRun).filter(
        ReconRun.tenant_id == tenant_id,
        ReconRun.run_date == run_date
    ).order_by(ReconRun.run_timestamp.desc()).first()

    if not report:
        raise HTTPException(status_code=404, detail="No report found for this date")

    return {
        "run_date": report.run_date.isoformat(),
        "total_trades": report.total_trades,
        "matched_trades": report.matched_trades,
        "unmatched_trades": report.unmatched_trades,
        "total_exceptions": report.total_exceptions,
        "by_severity": report.exceptions_by_severity,
        "by_domain": report.exceptions_by_domain,
        "by_rule": report.exceptions_by_rule,
        "execution_time_seconds": float(report.execution_time_seconds),
        "status": report.status
    }

@router.get("/dashboard/{tenant_id}")
async def get_recon_dashboard(
    tenant_id: str,
    days: int = Query(7), # Last 7 days by default
    db: Session = Depends(get_db)
):
    """
    Fetch dashboard summary: trends, aging, key metrics.
    """

    # Query recent runs
    recent_runs = db.query(ReconRun).filter(
        ReconRun.tenant_id == tenant_id,
        ReconRun.run_date >= (date.today() - timedelta(days=days))
    ).order_by(ReconRun.run_date.desc()).all()

    # Query open exceptions by age
    open_exceptions = db.query(Exception).filter(
        Exception.tenant_id == tenant_id,
        Exception.status == "OPEN"
    ).all()

    age_distribution = {
        "less_than_1_day": len([e for e in open_exceptions if e.minutes_open < 1440]),
        "1_to_3_days": len([e for e in open_exceptions if 1440 <= e.minutes_open < 4320]),
        "3_to_7_days": len([e for e in open_exceptions if 4320 <= e.minutes_open < 10080]),
        "more_than_7_days": len([e for e in open_exceptions if e.minutes_open >= 10080])
    }

    return {
        "recent_runs": [

```

```

        {
            "run_date": r.run_date.isoformat(),
            "total_exceptions": r.total_exceptions,
            "by_severity": r.exceptions_by_severity,
            "matched_trades": r.matched_trades,
            "total_trades": r.total_trades,
            "match_rate": (r.matched_trades / r.total_trades * 100) if r.total_trades
        }
    for r in recent_runs
],
"open_exceptions": {
    "total": len(open_exceptions),
    "by_severity": {
        "CRITICAL": len([e for e in open_exceptions if e.severity == "CRITICAL"]),
        "HIGH": len([e for e in open_exceptions if e.severity == "HIGH"]),
        "MEDIUM": len([e for e in open_exceptions if e.severity == "MEDIUM"]),
        "LOW": len([e for e in open_exceptions if e.severity == "LOW"])
    },
    "age_distribution": age_distribution
}
}

# ===== RULE CONFIGURATION ENDPOINTS =====

@router.get("/rules/{tenant_id}")
async def get_rule_config(
    tenant_id: str,
    db: Session = Depends(get_db)
):
    """
    Fetch active rule definitions for a tenant.
    """
    rules = db.query(RuleDefinition).filter(
        (RuleDefinition.tenant_id == tenant_id or RuleDefinition.tenant_id == None),
        RuleDefinition.is_active == True
    ).all()

    return {
        "rules": [
            {
                "rule_id": r.rule_id,
                "rule_name": r.rule_name,
                "tier": r.tier,
                "tolerance_abs": float(r.tolerance_abs) if r.tolerance_abs else None,
                "tolerance_pct": float(r.tolerance_pct) if r.tolerance_pct else None,
                "tolerance_days": r.tolerance_days,
                "severity_level": r.severity_level,
                "effective_from_date": r.effective_from_date.isoformat()
            }
            for r in rules
        ]
    }

@router.post("/rules/{tenant_id}/{rule_id}/update_tolerance")
async def update_rule_tolerance(
    tenant_id: str,

```

```

rule_id: str,
new_tolerance_abs: Optional[float] = None,
new_tolerance_pct: Optional[float] = None,
reason: str = "",
updated_by_user_id: str = "",
db: Session = Depends(get_db)
):
    """
    Update tolerance for a rule (for self-healing or manual adjustment).
    """
    # Create new version of rule with updated tolerance
    old_rule = db.query(RuleDefinition).filter(
        RuleDefinition.rule_id == rule_id,
        RuleDefinition.tenant_id == tenant_id,
        RuleDefinition.is_active == True
    ).order_by(RuleDefinition.rule_version.desc()).first()

    if not old_rule:
        raise HTTPException(status_code=404, detail="Rule not found")

    # Deactivate old version
    old_rule.is_active = False

    # Create new version
    new_rule = RuleDefinition(
        tenant_id=tenant_id,
        rule_id=rule_id,
        domain=old_rule.domain,
        rule_name=old_rule.rule_name,
        rule_description=old_rule.rule_description,
        rule_type=old_rule.rule_type,
        tier=old_rule.tier,
        should_live_in=old_rule.should_live_in,
        tolerance_abs=new_tolerance_abs or old_rule.tolerance_abs,
        tolerance_pct=new_tolerance_pct or old_rule.tolerance_pct,
        tolerance_days=old_rule.tolerance_days,
        tolerance_qty=old_rule.tolerance_qty,
        region_code=old_rule.region_code,
        is_active=True,
        effective_from_date=date.today(),
        severity_level=old_rule.severity_level,
        rule_version=old_rule.rule_version + 1,
        previous_tolerance_abs=old_rule.tolerance_abs,
        previous_tolerance_pct=old_rule.tolerance_pct,
        updated_by_user_id=updated_by_user_id,
        update_reason=reason
    )

    db.add(old_rule)
    db.add(new_rule)
    db.commit()

    return {
        "status": "updated",
        "rule_id": rule_id,
        "new_version": new_rule.rule_version,

```

```
        "new_tolerance_abs": float(new_rule.tolerance_abs) if new_rule.tolerance_abs else  
        "new_tolerance_pct": float(new_rule.tolerance_pct) if new_rule.tolerance_pct else  
    }
```

9.4 Integration with Existing Layers

A. Ingestion Layer → Rule Engine

```
# File: backend/ingestion/post_ingestion_hook.py  
  
# After file ingestion completes, trigger reconciliation  
  
async def on_file_ingestion_complete(file_upload_id: str, tenant_id: str, db: Session):  
    """  
    Hook called after LLM-based schema mapping completes.  
    Triggers incremental reconciliation on newly ingested data.  
    """  
    # Get newly ingested entities (last 5 minutes)  
    new_trades = db.query(Trade).filter(  
        Trade.tenant_id == tenant_id,  
        Trade.created_at >= datetime.now() - timedelta(minutes=5)  
    ).all()  
  
    new_cash_txns = db.query(CashTxn).filter(  
        CashTxn.tenant_id == tenant_id,  
        CashTxn.created_at >= datetime.now() - timedelta(minutes=5)  
    ).all()  
  
    # Run incremental recon on these entities  
    if new_trades or new_cash_txns:  
        engine = Phase1ReconciliationEngine(db, tenant_id, date.today())  
  
        # Only match these new trades  
        matched, breaks = engine._reconcile_trade_cash_exact(new_trades, new_cash_txns)  
  
        # Log exceptions for unmatched  
        engine._log_exceptions(breaks, rule_id="TRADE_CASH_001", severity="HIGH")  
  
        # Return summary to frontend (for quick feedback)  
        return {  
            "new_trades": len(new_trades),  
            "new_cash_txns": len(new_cash_txns),  
            "matched": len(matched),  
            "breaks": len(breaks)  
        }  
    }
```

B. Rule Engine → LLM Gateway (Phase 3)

```
# File: backend/rule_engine/ai_layer.py

from backend.llm.gateway import LLMGateway

class AIReasoningLayer:
    """
    Phase 3: Fuzzy matching and AI-driven resolution.
    Called only for residual breaks (unresolved by Tier-1 & 2).
    """

    def __init__(self, llm_gateway: LLMGateway, tenant_id: str):
        self.llm = llm_gateway
        self.tenant_id = tenant_id

    def reason_about_partial_fill(
        self,
        trade: Trade,
        candidate_cash_entries: List[CashTxn]
    ) -> Optional[str]: # Returns matched_cash_id or None
        """
        AI reasoning: Given a trade + multiple candidate cash entries,
        pick the best match using fuzzy logic.
        """
        prompt = f"""
        A broker trade needs to be matched to a bank cash entry. There are multiple candi

        Trade:
        - Symbol: {trade.symbol}
        - Qty: {trade.quantity}
        - Price: {trade.price}
        - Net Amount: {trade.net_amount}
        - Settlement Date: {trade.settlement_date}
        - Broker: {trade.broker}

        Candidate Cash Entries:
        """

        for i, cash in enumerate(candidate_cash_entries):
            prompt += f"""
            Candidate {i+1}:
            - Amount: {cash.amount}
            - Value Date: {cash.value_date}
            - Description: {cash.description}
            - Days from settlement: {(cash.value_date - trade.settlement_date).days}
            """

        prompt += """
        Which candidate (if any) matches this trade? Return JSON:
        {
            "best_match_index": <0, 1, 2, ... or null>,
            "confidence": <0.0-1.0>,
            "reasoning": "<explanation>"
        }
        """
```

```

        response = self.llm.call_openai_brain(
            prompt=prompt,
            system_role="You are a post-trade operations expert. Analyze the trade and cash entries to identify the source of an unmatched cash entry.",
            json_mode=True
        )

        if response and response.get("best_match_index") is not None:
            return candidate_cash_entries[response["best_match_index"]].id

        return None

def reason_about_unmatched_cash(
    self,
    cash: CashTxn,
    trade_candidates: List[Trade],
    corp_action_candidates: List[CorporateAction]
) -> dict: # Returns { source_type, source_id, confidence }
    """
    AI reasoning: What is the source of an unmatched cash entry?
    """

    prompt = f"""
    A bank cash entry cannot be matched to any trade or known corporate action.
    Help identify the source.

    Cash Entry:
    - Amount: {cash.amount}
    - Value Date: {cash.value_date}
    - Description: {cash.description}
    - Counterparty: {cash.counterparty}

    Possible Matching Trades (unmatched):
    """

    for t in trade_candidates[:5]: # Max 5 candidates
        prompt += f"\n - {t.broker} {t.side} {t.quantity} {t.symbol} @ {t.price}"

    prompt += f"""

    Possible Corp Actions:
    """

    for ca in corp_action_candidates[:3]:
        prompt += f"\n - {ca.type} {ca.symbol} on {ca.payment_date}"

    prompt += """

    Is this:
    1. A trade cash (link to one of the trades)?
    2. A dividend cash (link to a corp action)?
    3. A fee charge (custodian fee)?
    4. Unexplained (requires investigation)?

    Return JSON:
    {
        "source_type": "TRADE | DIVIDEND | FEE | UNEXPLAINED",

```

```

        "source_id": <id if matched, or null>,
        "confidence": <0.0-1.0>,
        "explanation": "<why>"
    }
    """

    response = self.llm.call_openai_brain(
        prompt=prompt,
        system_role="You are a settlement expert. Analyze the cash entry and determine the source and confidence.",
        json_mode=True
    )

    return response or { "source_type": "UNEXPLAINED", "confidence": 0.0 }

```

9.5 Cron Job Setup

```

# File: backend/cron/scheduled_jobs.py

from apscheduler.schedulers.background import BackgroundScheduler
from datetime import datetime, time

scheduler = BackgroundScheduler()

@scheduler.scheduled_job('cron', hour=20, minute=0) # 8 PM IST every day
def run_daily_reconciliation_job():
    """
    Daily 8 PM reconciliation run for all active tenants.
    """
    db = get_db_session()

    # Get all active tenants
    tenants = db.query(Tenant).filter(Tenant.is_active == True).all()

    for tenant in tenants:
        try:
            engine = Phase1ReconciliationEngine(db, tenant.id, date.today())
            result = engine.run_phase1_daily_reconciliation()

            logger.info(f"Reconciliation complete for {tenant.id}: {result}")

        except Exception as e:
            logger.error(f"Reconciliation failed for {tenant.id}: {e}")
            # Send alert to ops
            notify_slack(
                channel="#ops-alerts",
                message=f"⚠️ Reconciliation job failed for {tenant.id}: {e}"
            )

@scheduler.scheduled_job('cron', hour=23, minute=0) # 11 PM IST (nightly)
def run_self_healing_job():
    """
    Nightly self-healing: analyze learning events, propose rule updates.
    (Phase 3 feature)
    """

```

```

db = get_db_session()
tenants = db.query(Tenant).filter(Tenant.is_active == True).all()

for tenant in tenants:
    try:
        healing_agent = SelfHealingAgent(db, tenant.id)
        improvements = healing_agent.analyze_learning_events_batch(tenant.id, lookback)

        if improvements:
            logger.info(f"Self-healing: {len(improvements)} rule improvements proposed")
            # Store proposals for manual approval
            for improvement in improvements:
                db.add(RuleImprovement(
                    tenant_id=tenant.id,
                    rule_id=improvement.rule_id,
                    proposal=json.dumps(improvement),
                    status="PENDING_APPROVAL"
                ))
            db.commit()

    except Exception as e:
        logger.error(f"Self-healing job failed for {tenant.id}: {e}")

scheduler.start()

```

SUMMARY: WHAT YOU HAVE NOW

Phase 1 (4 weeks) deliverables:

1. **Rule Engine** (Python class in backend/rule_engine/phase1_engine.py)
 - 10 core rules (TRADE_CASH_001, TRADE_CASH_003, TRADE_CASH_005, NAV_VAL_001/002, etc.)
 - Exact matching + tolerance-based logic
 - ~85% auto-reconciliation rate
2. **Database Schema** (6 new tables)
 - exceptions (break queue)
 - learning_events (audit trail)
 - rule_definitions (parameterized rules)
 - recon_runs (reports)
 - broker_quirks (known issues)
 - fx_rates (for Phase 2+)
3. **API Endpoints**
 - POST /api/reconciliation/run_daily/{tenant_id}
 - GET /api/reconciliation/exceptions/{tenant_id}

- POST /api/reconciliation/exceptions/{id}/resolve
- GET /api/reconciliation/dashboard/{tenant_id}

4. Ops Workflows

- Daily 8 PM batch reconciliation
- Slack notifications for critical breaks
- Manual exception resolution + learning logging
- Exception aging / SLA tracking

5. Success Metrics

- 85% trades auto-reconciled (up from 0%)
- <4 hours ops per CRITICAL break
- <5% false positive rate

You now have:

- A production-ready Phase 1 roadmap (12 weeks total)
- Detailed rule specifications (110+ rules across 7 domains)
- Complete data model & schema
- API integration architecture
- Self-healing framework (for Phase 3)

Next steps:

1. Review with your team
2. Validate rule logic with real PMS/custodian data
3. Start Week 1 implementation (schema + skeleton engine)
4. Iterate based on real data patterns

Good luck! 🍀